IILM UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
IILM UNIVERSITY, GREATER NOIDA

PROJECT SYNOPSIS

On

Project ID – BTP2CSE096

Project Title - Email Spam Classifier

Using Machine Learning & Natural Language Processing

Submitted in partial fulfillment for the award of the degree of Minor Project

Session 2025-26
in
Computer Science and Engineering

By--- Yug Bhandari (2410031103)
Supriya (2410031272)
Kaninka Dhanai (2410031126)
Manmeet Singh (2410031271)
Shivam Saini (2410031279)

Under the guidance of --- Rahul Anjana
Assistant Professor

IILM UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
IILM UNIVERSITY, GREATER NOIDA, INDIA
March, 2026

INDEX

Section No.	Section	Page No.
1	Abstract	1
2	Introduction	1
3	Problem Statement	1
4	Dataset Description	2
5	Methodology	2
6	System Architecture	3
7	Technology Stack	3
8	Results & Key Findings	4
9	Future Scope	4
10	Conclusion	4
11	References	5

Signature of Supervisor: Rahul

Date: 27/03/26 27/03/26

PROJECT SYNOPSIS

Email Spam Classifier

Project Title	Email Spam Classifier
Domain	Machine Learning / NLP
Dataset	spamemail_dataset.csv (5,572 records)
Best Model	Multinomial Naive Bayes (Precision: 100%)
Tools Used	Python, Scikit-learn, NLTK, Streamlit, TF-IDF
Deployment	Streamlit Web Application

1. Abstract

The exponential rise of unsolicited electronic communications, commonly known as spam, poses significant threats to user productivity, digital security, and data privacy. This project presents the design, development, and deployment of an intelligent SMS/Email Spam Classifier using Natural Language Processing (NLP) and supervised Machine Learning techniques.

The system processes raw text messages, applies a multi-stage NLP preprocessing pipeline, and classifies each message as either spam or legitimate (ham). A comparative evaluation of twelve machine learning algorithms was conducted on the spamemail_dataset.csv dataset containing 5,572 labelled records. The Multinomial Naive Bayes classifier emerged as the optimal model, achieving 97.4% accuracy and a precision score of 100%, making it highly reliable for real-world deployment. The final system is deployed as an interactive Streamlit web application, enabling end-users to classify any email or SMS message in real time.

2. Introduction

Email and SMS remain the most widely used channels of digital communication globally. However, the ease of mass messaging has led to an overwhelming surge in spam — unsolicited, irrelevant, or malicious messages that clutter inboxes and often serve as vectors for phishing, fraud, and malware distribution.

Traditional rule-based spam filters rely on keyword matching and blacklists, which are easily circumvented by modern spam techniques. Machine learning-based approaches offer a more robust alternative by learning statistical patterns from labelled data, enabling accurate classification even for previously unseen spam messages. This project addresses the challenge of automated spam detection by building an end-to-end ML pipeline — from raw data ingestion and preprocessing to model training, evaluation, and deployment.

3. Problem Statement

Given a labelled dataset of email messages, build a text classification system that:

- Accurately identifies whether a given message is spam or ham (not spam).
- Achieves high precision to minimise false positives (legitimate messages flagged as spam).
- Processes and classifies new, unseen messages in real time through a user-facing application.

4. Dataset Description

The dataset used for this project is spamemail_dataset.csv, a well-known benchmark corpus for spam detection, sourced from Kaggle.

Attribute	Value	Notes
Total Records	5,572	Before deduplication
After Deduplication	5,157	415 duplicates removed
Ham (Legitimate)	4,516 (87.6%)	Class 0
Spam	641 (12.4%)	Class 1
Features	2 columns	Category, Message
Avg. Message Length	79.1 characters	Spam avg: ~150 chars

The dataset exhibits class imbalance, with spam messages comprising approximately 12.4% of the corpus. This characteristic was accounted for during model evaluation by prioritising precision as the key performance metric.

5. Methodology

The project follows an eight-stage pipeline as described below:

5.1 Data Cleaning

- Loaded the dataset and dropped duplicate entries (415 removed).
- Renamed columns to target and text for clarity.
- Applied Label Encoding: ham = 0, spam = 1.
- Verified no missing values were present.

5.2 Exploratory Data Analysis (EDA)

- Visualised class distribution via pie chart — confirmed significant class imbalance.
- Engineered three new features: num_characters, num_words, and num_sentences.
- Spam messages were found to be significantly longer on average (150 vs 72 characters).
- Plotted distribution histograms, pairplots, and a correlation heatmap.

5.3 Text Preprocessing

- Lowercased all text.
- Tokenised messages using regex-based word extraction.
- Removed stopwords and non-alphanumeric characters.
- Applied Porter Stemmer to reduce words to their root forms (e.g., 'winning' → 'win').
- Generated WordCloud visualisations for spam and ham corpora.
- Extracted and visualised the Top 30 most frequent words in each class.

5.4 Feature Engineering — TF-IDF Vectorisation

Transformed preprocessed text into numerical feature vectors using TF-IDF (Term Frequency – Inverse Document Frequency) with max_features = 3,000. The dataset was split 80/20 into training (4,125 samples) and test (1,032 samples) sets with random_state = 2.

5.5 Model Training & Comparison

Twelve classification models were trained and evaluated:

#	Algorithm	Accuracy	Precision
1	**Multinomial Naive Bayes ★ (Selected)**	97.4%	**100.0%**
2	Extra Trees Classifier	98.1%	100.0%
3	Random Forest	97.9%	100.0%
4	KNN Classifier	90.9%	100.0%
5	Bernoulli Naive Bayes	98.6%	99.1%
6	SVM (Sigmoid Kernel)	97.9%	97.3%
7	Gradient Boosting	95.4%	97.6%
8	Logistic Regression	96.2%	95.8%
9	Bagging Classifier	96.2%	89.2%
10	Decision Tree	94.2%	87.7%
11	AdaBoost	93.6%	87.6%

★ MultinomialNB was selected as the final model due to its perfect precision (100%), meaning zero legitimate messages were incorrectly flagged as spam — a critical requirement for production deployment.

6. System Architecture

The end-to-end pipeline consists of the following components:

Stage	Description
Input	Raw SMS / Email text message
Preprocessing	Lowercase → Tokenise → Remove stopwords → Stem (PorterStemmer)
Vectorisation	TF-IDF (max_features = 3,000)
Classification	MultinomialNB model (loaded from model.pkl)
Output	SPAM or HAM with confidence score
Interface	Streamlit web application (app.py)

7. Technology Stack

Category	Tool / Library	Purpose
Language	Python 3.10+	Core programming language
ML Framework	Scikit-learn	Model training, TF-IDF, evaluation
NLP	NLTK / Regex	Tokenisation, stemming, preprocessing
Visualisation	Matplotlib / Seaborn	EDA charts, confusion matrix
Word Cloud	WordCloud	Spam & ham corpus visualisation
Notebooks	Jupyter Notebook	Step-by-step experimental workflow
Web App	Streamlit	Real-time interactive deployment
Serialisation	Pickle	Save / load model and vectorizer
Data Handling	Pandas / NumPy	Data loading, manipulation, arrays

8. Results & Key Findings

- The best performing model is Multinomial Naive Bayes with 97.4% accuracy and 100% precision on the test set.
- 100% precision means no legitimate message was incorrectly classified as spam — the most critical requirement for production use.
- Spam messages were on average twice as long as legitimate messages (150 vs 72 characters), confirming message length as a useful discriminating feature.
- The most frequent spam keywords identified include: free, call, txt, prize, win, claim, urgent, reply, stop — consistent with real-world spam patterns.
- TF-IDF vectorisation with max_features = 3,000 provided sufficient vocabulary coverage without overfitting.

9. Future Scope

- Deep Learning: Replace TF-IDF + NB with LSTM or BERT-based transformer models for improved contextual understanding.
- Multilingual Support: Extend the classifier to handle non-English spam messages, particularly in Hindi and other regional languages.
- Active Learning: Implement feedback loops where users can flag misclassified messages to retrain the model continuously.
- Email Header Analysis: Incorporate metadata (sender domain, IP reputation, subject line entropy) as additional features.
- REST API: Expose the classifier as a RESTful API (FastAPI / Flask) for integration into email clients and mobile applications.
- Browser Extension: Build a Chrome/Firefox extension that classifies emails directly within Gmail or Outlook.

10. Conclusion

This project successfully demonstrates the application of Natural Language Processing and Machine Learning to the real-world problem of spam detection. By systematically comparing twelve classification algorithms and deploying the best-performing model through an interactive Streamlit application, the project delivers a complete, production-ready solution.

The Multinomial Naive Bayes model's perfect precision of 100% makes it particularly well-suited for deployment scenarios where false positives (legitimate messages flagged as spam) must be eliminated. The modular pipeline architecture ensures easy extension to new datasets, additional features, or more sophisticated deep learning approaches in the future. This work provides a strong foundation for further research into adaptive, multilingual, and context-aware spam detection systems.

11. References

1. Hands on Machine Learning with Scikit-Learn Book by Aurelien Georon.

2. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825-2830.

3. Bird, S., Klein, E., Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

4. Streamlit Inc. (2024). Streamlit Documentation. https://docs.streamlit.io

5.https://www.researchgate.net/publication/391732142_Email_Spam_Detection_Using_Machine_Learning